

Weekly Learning Debrief

MIS410 — Data Analytics Certificate | Week 5

Topics: SQL text functions, NLTK preprocessing, VADER sentiment analysis, LDA topic modeling, full text analysis pipeline

Reading: SQL for Text, and Where SQL Stops

SQL can apply explicit, rule-based operations to text at scale (splitting, pattern matching), but has real limits: small/qualitative tasks are often better done by a human, fast full-text search belongs to tools like Elasticsearch or Splunk, and anything requiring true language understanding (sentiment, part-of-speech tagging, language generation) belongs to Python's ML/NLP libraries. Splunk in particular is widely used in security/SIEM work — worth remembering for the cybersecurity track.

- `split_part()` is a blind mechanical cut at a chosen delimiter — not an intelligent identification of meaning. It works because report templates repeat fixed labels (“Occurred :”, “Location:”) in the same spot every time.
- `LIKE` matches a pattern using `%` (any amount of characters) and `_` (exactly one character); `ILIKE` is identical but ignores capitalization.
- `Regex` is a far more powerful pattern-matching mini-language than `LIKE`'s wildcards — used when a simple `% / _` pattern isn't specific enough.

Assignment: SQL + Python Text Analysis on Product Reviews

Built a full pipeline on a two-table dataset (products, reviews): SQL extraction/filtering/aggregation, then Python for preprocessing, sentiment analysis, topic modeling, and three visualizations.

Tool	What it does
<code>split_part(text, delim, n)</code>	SQL: cuts text at a delimiter, returns the nth chunk — a mechanical cut, not an intelligent identification
<code>LIKE / ILIKE</code> with <code>%</code> and <code>_</code>	SQL: pattern matching. <code>%</code> = any amount of characters, <code>_</code> = exactly one. <code>ILIKE</code> ignores capitalization
Regex	A mini-language for describing complex text patterns — <code>LIKE</code> 's much more powerful cousin
<code>word_tokenize()</code>	NLTK: splits text into individual word tokens
<code>stopwords.words()</code>	NLTK: the list of common filler words (the, is, and) to filter out
<code>WordNetLemmatizer()</code>	NLTK: reduces a word to its dictionary base form (running → run)
<code>SentimentIntensityAnalyzer()</code> (VADER)	NLTK: scores text as positive/negative/neutral using a pre-scored word lexicon plus rules
<code>CountVectorizer()</code>	scikit-learn: converts text into a word-count matrix (rows = documents, columns = words)
<code>LatentDirichletAllocation()</code>	scikit-learn: LDA — groups words that tend to co-occur into a set number of topics

- **SQL (Task 1):** date-range filtering, rating filtering, `AVG(rating)` joined with product category, `COUNT(review_id)` grouped by category — review counts ranged from 40 (Electronics) to 111 (Toys), a notable volume imbalance.
- **Preprocessing (Task 2):** tokenized, removed stop words, lemmatized every review using NLTK. Required an extra one-time download (“punkt_tab”) beyond the standard “punkt” resource due to a recent NLTK version change.

- **Sentiment analysis:** used VADER's compound score, thresholded at ± 0.05 into positive/negative/neutral labels.
- **Topic modeling:** used LDA with 5 topics via CountVectorizer + LatentDirichletAllocation, printing the top 10 words per topic.
- **Visualizations:** a stacked sentiment bar chart by product, a word cloud (dominated by “item,” “arrived,” “damaged,” “good,” “value”), and average rating by category.
- **Data limitation noted:** several reviews share near-identical text (e.g. “Item arrived damaged” repeats often), indicating a smaller, templated dataset rather than fully varied real-world reviews — worth flagging when interpreting sentiment/topic results.

Deep Dive: How VADER Actually Works

VADER has no real language understanding — it's a pre-scored lookup table with adjustment rules layered on top.

- **The lexicon:** ~7,500 words, each hand-scored from about -4 to +4 (e.g. “great” $\approx +3.1$, “terrible” ≈ -2.9). Scoring a sentence starts by looking up every recognized word.
- **Rules layered on top:** negation (“not good” flips/dampens the score), intensifiers (“very good,” “GOOD,” “!!!” boost it), built specifically to handle informal/social-media-style text.
- **The compound score:** all adjusted word scores are summed, then normalized into a clean -1 to +1 range.
- **Real limitation:** zero understanding of context or sarcasm (“oh great, another delay” scores positive). This is exactly the gap that modern transformer models like BERT are built to close.

Deep Dive: How LDA Actually Works

Analogy: every review is a dish made from a blend of 5 unseen “spice jars” (topics). LDA never sees the jars directly — it only sees the finished dishes and works backward to guess the blend.

- LDA starts by randomly assigning every word in every review to one of the topics — pure noise at first.
- It then repeatedly revisits each word, asking two questions: what topics do the OTHER words in this same review currently belong to, and what topic has this exact word been assigned to everywhere else in the dataset?
- Each pass nudges assignments to be slightly more consistent. Words that truly tend to co-occur (“shipping,” “arrived,” “damaged,” “box”) reinforce each other's grouping over thousands of iterations, eventually settling into stable clusters.
- LDA never knows what a topic “means” — it only finds words that cluster together. Reading the top words per topic and naming it is the analyst's job, not LDA's.
- **n_components** is a number YOU choose — LDA doesn't know how many topics genuinely exist, it forces the data into however many groups it's told to find.
- Honest limitation: LDA can produce noisy or overlapping topics on a small, templated dataset (like this week's reviews) — it performs best on large, genuinely varied text collections.

Next steps: bring this same format back each week to track growth toward the Mod 8 goal of working independently.